i am going to provide you with aim of practical you have to give me practical writeups for the AQC in this format
1. Aim
2. System requirements
3. Theory
4. Algorithm
5. Conclusion

Got it 👍 — send me the **aim of the practical**, and I'll generate a clean AQC write-up in this format:

1. Aim
2. System Requirements
3. Theory
4. Algorithm
5. Conclusion

If you have any specific constraints (language like Python/C++, word limit, or college format), tell me along with the aim.

1. Implement and verify the Bernstein–Vazirani algorithm in Colab

Here's a clean AQC practical write-up for your aim:

---

1. Aim

To implement and verify the **Bernstein–Vazirani algorithm** using **Google Colab** and simulate it with a quantum computing framework.

---

2. System Requirements

- **Hardware:**

    - Computer/Laptop with minimum 4GB RAM
- **Software:**
    - Google Colab (web-based Python environment)
    - Python (version 3.x)
    - Qiskit library
    - Internet connection

---

3. Theory

The **Bernstein–Vazirani algorithm** is a quantum algorithm used to determine a hidden binary string s encoded in a function:

$$f(x) = s \cdot x \pmod 2$$

where x and s are binary strings, and $\cdot$ denotes the bitwise dot product.

Key Concept:

- Classically, determining the string s requires **n queries** to the oracle.
- Quantum mechanically, this algorithm finds s in **just one query**, demonstrating **quantum parallelism**.

Working Principle:

1. Initialize qubits.
2. Apply Hadamard gates to create superposition.
3. Use an oracle that encodes the hidden string.
4. Apply Hadamard transformation again.
5. Measure the output to obtain the hidden string directly.

---

4. Algorithm

Step 1: Import required libraries (Qiskit).
Step 2: Define the number of qubits n.
Step 3: Initialize a quantum circuit with $n + 1$ qubits (including one ancilla qubit).
Step 4: Apply Hadamard gates to all input qubits.
Step 5: Prepare the ancilla qubit in state $|1\rangle$ and apply Hadamard gate.
Step 6: Construct the oracle corresponding to the hidden binary string s.
Step 7: Apply the oracle to the circuit.
Step 8: Apply Hadamard gates again to the input qubits.
Step 9: Measure the input qubits.
Step 10: Run the circuit on a simulator and observe the output.

5. Conclusion

The **Bernstein–Vazirani algorithm** was successfully implemented using **Qiskit** in **Google Colab**. The simulation result correctly revealed the hidden binary string in a single execution, verifying the efficiency of quantum computation over classical methods for this problem.

2. Execute Simon's algorithm and observe the hidden bit string

Here's your AQC practical write-up:

---

1. Aim

To implement and execute **Simon's algorithm** and observe the hidden bit string using a quantum simulator.

---

2. System Requirements

- **Hardware:**
  - Computer/Laptop with minimum 4GB RAM
- **Software:**
  - Google Colab
  - Python (3.x)
  - Qiskit
  - Internet connection

---

3. Theory

Simon's algorithm is one of the earliest quantum algorithms that demonstrates an exponential speedup over classical algorithms.

Problem Statement:

Given a function $f(x)$ such that:

$$f(x) = f(x \oplus s)$$

where:

- s is a hidden binary string
- $\oplus$ denotes bitwise XOR

The goal is to determine the hidden string s.

Key Concept:

- Classically, solving this problem requires exponential time.
- Quantumly, Simon's algorithm finds s in **polynomial time**.

Working Principle:

1. Initialize two registers of qubits.
2. Apply Hadamard gates to create superposition.
3. Apply the oracle representing the function $f(x)$.
4. Measure the output register (collapses entanglement).
5. Apply Hadamard gates again to input qubits.
6. Measure input qubits to obtain equations of the form:

$$y \cdot s = 0 \pmod 2$$

7. Solve these linear equations to determine the hidden string s.

---

4. Algorithm

Step 1: Import Qiskit libraries.
Step 2: Define number of qubits n and hidden string s.
Step 3: Initialize a quantum circuit with $2n$ qubits (input + output registers).
Step 4: Apply Hadamard gates to input qubits.
Step 5: Implement the oracle for Simon's function.
Step 6: Measure the output register.
Step 7: Apply Hadamard gates again to input qubits.
Step 8: Measure input qubits to obtain bit strings y.
Step 9: Repeat execution multiple times to collect enough equations.
Step 10: Solve the system of linear equations (mod 2) to find s.

---

5. Conclusion

The **Simon's algorithm** was successfully executed using **Qiskit** in **Google Colab**. The results obtained from measurements were used to form linear equations, which correctly revealed

the hidden bit string s, demonstrating the quantum advantage over classical computation.

3. Implement Grover's search algorithm for a 2-qubit search problem

Here's your AQC practical write-up:

---

1. Aim

To implement **Grover's algorithm** for a 2-qubit search problem and identify the marked element using a quantum simulator.

---

2. System Requirements

- **Hardware:**
  - Computer/Laptop with minimum 4GB RAM
- **Software:**
  - Google Colab
  - Python (3.x)
  - Qiskit
  - Internet connection

---

3. Theory

Grover's algorithm is a quantum algorithm used to search an unsorted database efficiently.

Problem Statement:

Given a search space of $N = 2^n$ elements, find a marked element using an oracle function.

Key Concept:

- Classical search takes **O(N)** time.
- Grover's algorithm reduces it to **O(\sqrt{N})**, providing quadratic speedup.

Working Principle:

1. Initialize qubits in equal superposition using Hadamard gates.
2. Apply the **oracle** to mark the desired state (phase inversion).
3. Apply the **diffusion operator** (inversion about the mean).
4. Repeat the oracle and diffusion steps optimal number of times.
5. Measure the qubits to obtain the marked state.

For 2-Qubit Case:

- Total states = $2^2 = 4$
- Only **1 iteration** of Grover's operator is sufficient to find the marked element with high probability.

---

4. Algorithm

Step 1: Import Qiskit libraries.
Step 2: Initialize a quantum circuit with 2 qubits.
Step 3: Apply Hadamard gates to both qubits to create superposition.
Step 4: Design the oracle to mark the desired state (e.g., $|11\rangle$).
Step 5: Apply the diffusion operator:

- Apply Hadamard gates
- Apply X gates
- Apply multi-controlled Z gate
- Apply X gates again
- Apply Hadamard gates again

Step 6: Measure the qubits.
Step 7: Execute the circuit on a simulator.
Step 8: Observe the output state with the highest probability.

---

5. Conclusion

The **Grover's algorithm** was successfully implemented for a 2-qubit system using **Qiskit** in **Google Colab**. The marked state was obtained with high probability after a single iteration, confirming the effectiveness and speedup provided by Grover's search algorithm over classical methods.

4. Extend Grover's algorithm for a 3-qubit oracle and analyze iterations

Here's your AQC practical write-up:

---

1. Aim

To extend **Grover's algorithm** for a 3-qubit oracle and analyze the number of iterations required to successfully identify the marked state.

---

2. System Requirements

- **Hardware:**
  - Computer/Laptop with minimum 4GB RAM
- **Software:**
  - Google Colab
  - Python (3.x)
  - Qiskit
  - Internet connection

---

3. Theory

Grover's algorithm provides a quadratic speedup for searching an unsorted database.

Problem Statement:

For a 3-qubit system, the search space contains:

$$N = 2^3 = 8 \text{ states}$$

The goal is to find one marked state using an oracle.

Key Concept:

- Grover's algorithm amplifies the probability amplitude of the marked state.
- The optimal number of iterations is approximately:

$$R \approx \frac{\pi}{4}\sqrt{N}$$

For $N = 8$:

$$R \approx \frac{\pi}{4}\sqrt{8} \approx 2$$

Thus, **2 iterations** are optimal.

Working Principle:

1. Initialize all qubits in superposition.
2. Apply oracle to mark the target state (phase flip).
3. Apply diffusion operator to amplify probability.
4. Repeat oracle + diffusion for optimal iterations.
5. Measure qubits to obtain the marked state.

Analysis:

- **1 iteration:** Moderate probability of correct result
- **2 iterations:** Maximum probability (optimal)
- **More than 2:** Probability decreases due to over-rotation

---

4. Algorithm

Step 1: Import Qiskit libraries.
Step 2: Initialize a quantum circuit with 3 qubits.
Step 3: Apply Hadamard gates to all qubits.
Step 4: Construct the oracle for the marked state (e.g., $|101\rangle$).
Step 5: Apply the diffusion operator:

- Apply Hadamard gates
- Apply X gates
- Apply multi-controlled Z gate
- Apply X gates
- Apply Hadamard gates

Step 6: Repeat Steps 4 and 5 for **2 iterations**.
Step 7: Measure all qubits.
Step 8: Execute the circuit on a simulator.
Step 9: Analyze output probabilities for different iteration counts.

---

5. Conclusion

The extended implementation of **Grover's algorithm** for a 3-qubit system was successfully performed using **Qiskit** in **Google Colab**. It was observed that performing **2 iterations**

yields the highest probability of measuring the marked state, confirming the theoretical prediction and demonstrating the importance of optimal iteration count in Grover's algorithm.

5. Implement the Quantum Fourier Transform (QFT) and its inverse using Qiskit

Here's your AQC practical write-up:

---

1. Aim

To implement the **Quantum Fourier Transform** and its inverse using **Qiskit** and verify their correctness on a quantum simulator.

---

2. System Requirements

- **Hardware:**
  - Computer/Laptop with minimum 4GB RAM
- **Software:**
  - Google Colab
  - Python (3.x)
  - Qiskit
  - Internet connection

---

3. Theory

The **Quantum Fourier Transform** is the quantum analogue of the classical discrete Fourier transform and is widely used in many quantum algorithms such as phase estimation and factoring.

Definition:

For an n-qubit quantum state, QFT transforms a basis state $|x\rangle$ into:

$$|x\rangle \to \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi i x y/2^n} |y\rangle$$

Key Concept:

- QFT converts quantum states into their frequency domain.
- It can be implemented efficiently using **Hadamard gates** and **controlled phase rotation gates**.
- The **inverse QFT (QFT^{-1})** reverses the transformation and recovers the original state.

Working Principle:

1. Apply Hadamard gate to the first qubit.
2. Apply controlled phase rotations with decreasing angles.
3. Repeat for all qubits.
4. Swap qubits to reverse order.
5. For inverse QFT, apply reverse operations with negative phase angles.

---

4. Algorithm

QFT Implementation:

Step 1: Import Qiskit libraries.
Step 2: Initialize a quantum circuit with n qubits.
Step 3: For each qubit i:

- Apply Hadamard gate
- Apply controlled phase rotations with angles $\frac{\pi}{2^k}$

Step 4: Swap qubits to reverse their order.

Inverse QFT Implementation:

Step 5: Apply swap gates again.
Step 6: Apply inverse controlled phase rotations (negative angles).
Step 7: Apply Hadamard gates in reverse order.

Verification:

Step 8: Apply QFT followed by inverse QFT.
Step 9: Measure the qubits.
Step 10: Verify that the output matches the input state.

---

5. Conclusion

The **Quantum Fourier Transform** and its inverse were successfully implemented using **Qiskit** in **Google Colab**. The correctness was verified by applying QFT followed by its inverse, which successfully reproduced the original input state, demonstrating the accuracy and reversibility of the transformation.

6. Demonstrate Shor's algorithm for periodic factoring of small integers (e.g., 15 = 3 × 5)

Here's your AQC practical write-up:

---

1. Aim

To demonstrate **Shor's algorithm** for periodic factoring of small integers (e.g., $15 = 3 \times 5$) using a quantum simulator.

---

2. System Requirements

- **Hardware:**
  - Computer/Laptop with minimum 4GB RAM
- **Software:**
  - Google Colab
  - Python (3.x)
  - Qiskit
  - Internet connection

---

3. Theory

Shor's algorithm is a quantum algorithm used to factor large integers efficiently, providing exponential speedup over classical algorithms.

Problem Statement:

Factor a composite number N (e.g., $N = 15$) into its prime factors.

Key Concept:

- Classical factoring is computationally hard for large numbers.
- Shor's algorithm reduces the problem to **period finding**, which is efficiently solved using quantum computation.

Working Principle:

1. Choose a random integer a such that $1 < a < N$.
2. Compute $\gcd(a, N)$. If not 1, a factor is found.
3. Find the period r of the function:

$$f(x) = a^x \mod N$$

4. If r is even, compute:

$$\gcd(a^{r/2} \pm 1, N)$$

5. This yields non-trivial factors of N.

Quantum Role:

- The period r is determined using **Quantum Phase Estimation**, which internally uses the **Quantum Fourier Transform**.

---

4. Algorithm

Step 1: Import Qiskit libraries.
Step 2: Choose integer $N = 15$.
Step 3: Select a random a such that $\gcd(a, 15) = 1$.
Step 4: Initialize quantum registers for phase estimation.
Step 5: Apply Hadamard gates to create superposition.
Step 6: Implement modular exponentiation $a^x \mod N$.
Step 7: Apply inverse QFT to extract periodicity.
Step 8: Measure the quantum register to obtain result.
Step 9: Classically compute period r from measurement.
Step 10: Use r to compute factors using GCD.

---

5. Conclusion

The **Shor's algorithm** was successfully demonstrated for factoring the integer 15 using **Qiskit** in **Google Colab**. The algorithm correctly identified the factors 3 and 5, illustrating the power of quantum computation in solving classically hard problems like integer factorization.

7. Create a simple Quantum Support Vector Machine (QSVM) classifier using Qiskit Machine Learning

Here's your AQC practical write-up:

---

1. Aim

To create a simple **Quantum Support Vector Machine** classifier using **Qiskit Machine Learning** and evaluate its performance on a sample dataset.

---

2. System Requirements

- **Hardware:**
  - Computer/Laptop with minimum 4GB RAM
- **Software:**
  - Google Colab
  - Python (3.x)
  - Qiskit
  - Qiskit Machine Learning
  - Internet connection

---

3. Theory

The **Quantum Support Vector Machine** is a quantum-enhanced version of the classical Support Vector Machine used for classification tasks.

Key Concept:

- Classical SVM uses kernel functions to map data into higher-dimensional space.
- QSVM uses a **quantum feature map** to encode classical data into quantum states.
- It computes similarity between data points using a **quantum kernel**, which may provide advantages for complex datasets.

Working Principle:

1. Encode classical data into quantum states using a feature map.
2. Compute the quantum kernel (inner product of quantum states).
3. Use this kernel in an SVM classifier.
4. Train the model on labeled data.
5. Predict labels for unseen data.

Advantages:

- Potential to handle high-dimensional data efficiently
- Exploits quantum state space for feature representation

---

4. Algorithm

Step 1: Import required libraries from Qiskit and Qiskit Machine Learning.
Step 2: Load or generate a sample dataset (e.g., binary classification).
Step 3: Split dataset into training and testing sets.
Step 4: Define a quantum feature map (e.g., ZZFeatureMap).
Step 5: Create a quantum kernel using the feature map.
Step 6: Initialize the QSVM classifier.
Step 7: Train the QSVM model on training data.
Step 8: Test the model on test data.
Step 9: Evaluate performance using accuracy metrics.

---

5. Conclusion

The **Quantum Support Vector Machine** was successfully implemented using **Qiskit Machine Learning** in **Google Colab**. The classifier was able to learn from the dataset and make predictions, demonstrating the application of quantum computing in machine learning tasks.

8. Simulate the BB84 Quantum Key Distribution protocol using Qiskit circuits in Colab

Here's your AQC practical write-up:

---

1. Aim

To simulate the **BB84 protocol** using **Qiskit** circuits in **Google Colab** and demonstrate secure quantum key exchange.

---

2. System Requirements

- **Hardware:**
  - Computer/Laptop with minimum 4GB RAM
- **Software:**
  - Google Colab
  - Python (3.x)
  - Qiskit
  - Internet connection

---

3. Theory

The **BB84 protocol** is a quantum cryptographic protocol used to securely distribute encryption keys between two parties, commonly referred to as Alice and Bob.

Key Concept:

- Based on principles of **quantum mechanics** such as superposition and measurement.
- Any eavesdropping attempt (by an attacker, Eve) disturbs the quantum states and can be detected.

Working Principle:

1. Alice prepares qubits in random bases (rectilinear "+" or diagonal "×").
2. Alice sends qubits to Bob.
3. Bob measures each qubit in randomly chosen bases.
4. Alice and Bob publicly compare their bases (not values).
5. They keep only the bits where their bases match (sifted key).
6. A subset of bits is compared to detect eavesdropping.
7. If no intrusion is detected, the remaining bits form a secure key.

Security Feature:

- Measurement changes quantum states, so interception is detectable.

---

4. Algorithm

Step 1: Import Qiskit libraries.
Step 2: Generate random bit string and random bases for Alice.
Step 3: Encode bits into qubits using corresponding bases.
Step 4: Simulate transmission of qubits to Bob.
Step 5: Generate random bases for Bob.
Step 6: Bob measures qubits using his bases.
Step 7: Compare bases between Alice and Bob.
Step 8: Extract matching bits to form sifted key.
Step 9: Compare a subset of bits to check for eavesdropping.
Step 10: Generate final secure key if no interference is detected.

---

5. Conclusion

The **BB84 protocol** was successfully simulated using **Qiskit** in **Google Colab**. The protocol demonstrated secure key exchange and the ability to detect eavesdropping, highlighting the practical application of quantum mechanics in cryptography.

https://chatgpt.com/share/69eb700d-4fcc-8323-8352-c5688aca9bff